Lerunbot: Lerobot Runs

Zeyang Zhang
zhangzey24@mails.tsinghua.edu.cn

Chengwei Liang
lcw24@mails.tsinghua.edu.cn

Yiqi Dong
dongyq24@mails.tsinghua.edu.cn

Abstract

We present Lerunbot, a system that enables a standard 6-DoF robotic arm to walk on an unpowered base by pushing against the ground. To achieve this, we design a flexible control framework and a robust Reinforcement Learning (RL) training system that supports both simulation and real-world RL. We train a control policy in simulation and transfer it to the physical robot. Our approach is validated through real-world experiments, including a 10-meter shuttle run and obstacle avoidance, demonstrating the first unmodified robotic arm capable of stable and fast walking locomotion.

1 Introduction

In this project, we introduce **Lerunbot**, an innovative embodied AI system engineered to endow a standard 6 degrees-of-freedom (DoF) robotic arm with a remarkable and unconventional capability: the ability to ***walk***.

Specifically, Lerunbot integrates a Lerobot SO-101 arm [3, 5] with a mobile, unpowered base. By intelligently leveraging its end-effector to push against the ground, the arm is able to propel the entire robot forward, achieving walking locomotion. This approach transforms the arm from a stationary manipulator into an active driving mechanism, moving the robot to new locations through a sequence of coordinated pushes and lifts.

Lerunbot represents a significant milestone in robotic mobility: it is the first documented instance in which a single, unmodified robotic arm has been demonstrated to achieve stable walking locomotion without relying on any powered wheels. Furthermore, it is the first robotic arm to successfully complete a 10-meter shuttle run under its own "leg" power, with zero powered wheel assistance—a feat that crowns it as the world's "fastest" walking robotic arm to date.

2 Hardware Configuration

As shown in Figure 2, the system mounts a Lerobot SO-101 arm on a mobile tray. We use ball universal wheels to allow smooth movement in all directions and minimize noise. Initial experiments with standard stem swivel caster revealed significant trajectory variance due to wheel alignment issues, where the wheels would occasionally bind if not aligned with the motion vector.

The platform is equipped with a 1080P USB camera to enable real-world training, VLM integration, and teleoperation.

Onboard computing is provided by a Raspberry Pi 4B powered by a portable power bank, making the system fully self-contained and accessible via SSH. A UPS is placed between the power bank and system components to allow for hot-swapping power sources, theoretically allowing for unlimited continuous operation for extended training and deployment sessions.

Preprint.

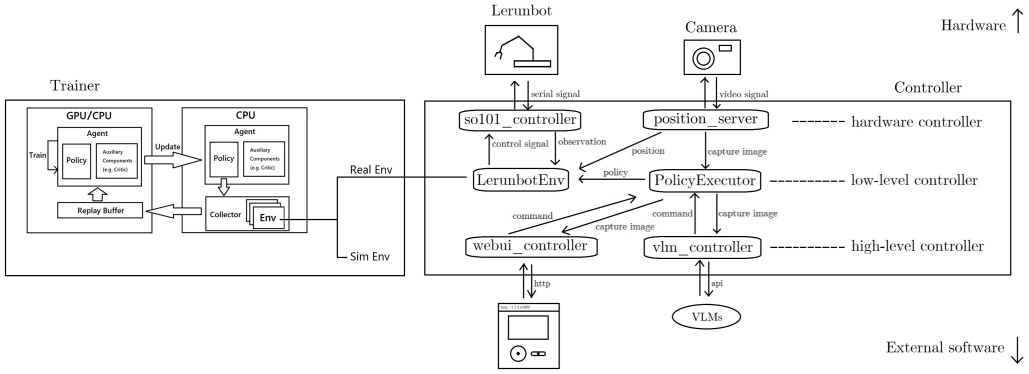


Figure 1: Lerunbot system software architecture.

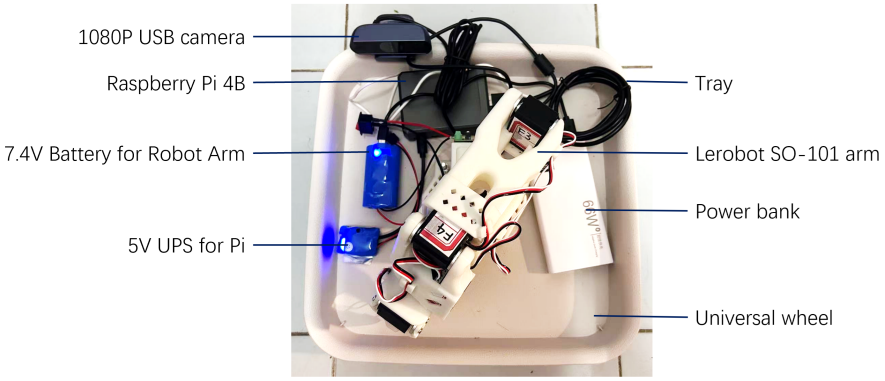


Figure 2: Lerunbot hardware setup.

3 RL Training System

We use Reinforcement Learning (RL) to learn the control policy. Inspired by SERL [6], we built a general RL training system compatible with both simulated and real-world environments, supporting both synchronous and asynchronous execution. The system is built upon TorchRL [2], which provides the core components.

As illustrated in the left part of Figure 1, we divide the training system into two separate processes: a training process and a rollout process. This separation allows for asynchronous operations. In asynchronous mode, the training process updates policy weights and periodically sends them to the rollout process, while the rollout process collects data into a shared replay buffer. For on-policy-like algorithms, training speed is automatically controlled. In synchronous mode, training and rollout run sequentially.

4 Simulation

Our approach involves initially training a policy in simulation followed by transfering to the real world. This section details how we constructed the simulation environment and trained the policy.

4.1 Simulation Environment

We build the simulation environment using MuJoCo [9] to mimic the physical setup. As shown in Figure 3, the SO-101 arm is mounted on a cuboid base approximating the wheeled tray. Physical parameters such as size, mass, and friction are tuned to match the real-world hardware. We did not implement visual sensors in the simulation because the policy relies on proprioception.

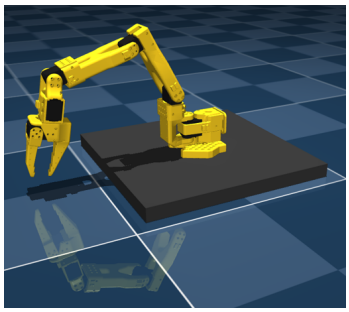


Figure 3: Simulated robot.

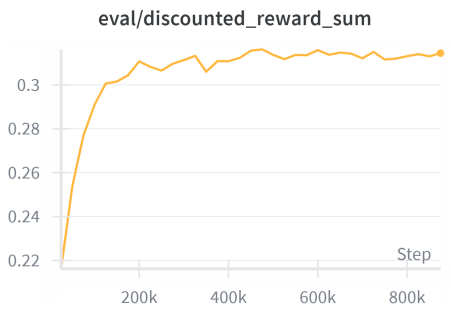


Figure 4: Evaluation performance.

At the start of each episode, the robot is initialized at the origin. A target position and orientation is sampled within a sector of 0.2m radius and 60-degree span for position, and $\pm 30^\circ$ yaw for orientation. While not the theoretical limit, this range was deemed sufficient for this scope. The objective is to reach the target within a fixed number of steps. The state space consists of the target position and orientation, base and joint generalized positions and velocities, the previous action, and the current timestamp. The action space defines target joint positions. We manually restricted the movement to only joints 0, 1, 2, and 3. To prevent policy collapse to a stationary local optimum, we set the initial configuration such that the end-effector is near the ground, as shown in Figure 3, helping early ground interaction during exploration.

The reward function combines progress towards the target, a smoothness penalty, and a tracking penalty minimizing the error between target and actual joint positions. Our rationale for the tracking penalty is based on the system's kinematics: since locomotion is driven by the end-effector pushing against the ground, assuming continuous contact and infinite friction, the entire robot's motion is strictly determined by the changes in joint angles. Therefore, by encouraging the policy to output target joint angles that match the actual realized angles, we ensure the learned control commands directly correspond to the physical state required for movement. This alignment improves the stability of the policy and enhances its ability to generalize across different environments. Additionally, a terminal penalty encourages the robot to return to the initial pose at the end of an episode. This ensures that the robot is physically close to its starting configuration before the next episode begins, preventing large, abrupt maneuvers during the reset phase that could cause unintended base movement. This smooth transition also facilitates the learning of cyclic, repeatable locomotion gait.

4.2 Experiments

We used Soft Actor-Critic (SAC) [4] for policy training in the simulation. Both the policy and Q-networks were implemented as 2-layer Multi-Layer Perceptrons (MLPs) with 512 hidden units per layer. Training was done in asynchronous mode, collecting data from parallel environments. The process consisted of 10^6 update steps and was completed within 30 minutes on a single NVIDIA RTX 4090 GPU, significantly accelerating the development cycle. Figure 4 shows the evaluation performance during training, while Figure 5 shows a representative trajectory of the learned behavior.

5 Real World

5.1 Control Pipeline

As shown in the right part of Figure 1, the control architecture is organized into three levels: *hardware layer*, *low-level controller*, and *high-level controller*.

The **Hardware layer** includes two main modules: `so101_controller`, which connects directly with the Lerobot SO-101 arm via serial communication for joint control and state readings; and `position_server`, which is responsible for base pose estimation. Currently, this is implemented by managing video streaming from the onboard camera and performing chessboard detection, as shown in Figure 6.

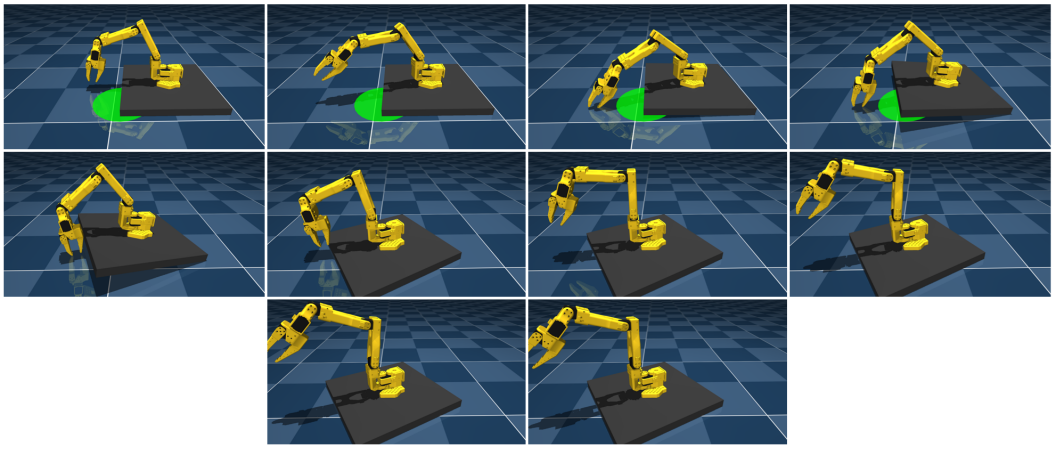
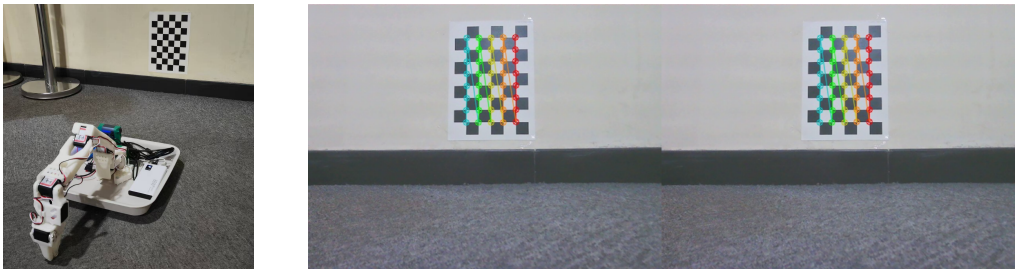


Figure 5: Example rollout in simulation. The target is to turn left 30 degrees and walk 0.2 m.



(a) Chessboard detection. (b) Pose estimation by comparing two frames of the chessboard.

Figure 6: Base pose estimation using camera and chessboard pattern.

The **Low-level controller** executes basic movement skills that correspond directly to the simulation training targets: reaching a specific relative position and orientation. These skills manifest as standard maneuvers like forward translation and turning. The core component, `LerunbotEnv`, implements the standard OpenAI Gym interface, abstracting hardware interactions for higher-level modules. We implemented multiple `PolicyExecutors` interacting with `LerunbotEnv`: `manual_controller` for replaying demonstration trajectories, and `nn_controller` for deploying trained RL policies.

The **High-level controller** issues navigation directives to the low-level system. Currently, this includes a `vlm_controller`, which translates natural language instructions via a Vision-Language Model (VLM) into navigational waypoints, and a `webui_controller`, offering a browser-based interface for direct teleoperation.

Hardware layer operates as persistent background daemons on the onboard computer. In contrast, control layers can be started on-demand and deployed remotely. Inter-module communication is handled by ZMQ messaging, ensuring modularity and deployment flexibility.

5.2 Low-level Control

The system supports three low-level control strategies: *trajectory replay*, *planning-based control*, and *open-loop neural control*.

Trajectory Replay: This method executes pre-recorded demonstration trajectories for basic primitives like forward walking and turning. Demonstrations are collected by manually guiding the arm and keyframing joint configurations. Additionally, trajectories generated in simulation can be directly transfered for replay on the physical hardware.

Planning-based Control: This approach uses motion planning to generate joint trajectories. We generate optimal trajectories in the MuJoCo simulation using the learned RL policy and then execute these plans on the physical robot via the replay mechanism.

Open-loop Neural Control: This method uses a neural network to generate a complete sequence of joint actions for a given target, without requiring real-time environmental feedback.

Currently, to obtain this controller, we first rollout the trained RL policy in the simulation to collect a dataset of demonstration trajectories. We then use Behavior Cloning (BC) to train an open-loop network to mimic these trajectories.

We further attempt to improve this network using Reinforcement Learning directly in the real world. In this setting, the observation consists solely of the target, and the policy outputs the entire trajectory. The reward is defined as the cumulative sum of the single-step rewards used in simulation. Calculating this is feasible: the progress reward relies only on the base pose at the start and end of the trajectory, which is obtained via the camera; the smoothness penalty depends only on actions; and the tracking penalty uses actual joint positions, which have acceptable measurement variance compared to joint velocities.

We employ the Proximal Policy Optimization (PPO) [7] algorithm. We initialize the policy with the BC-trained network. To initialize the value network, which estimates the expected return given a target, we fit it to data collected by the BC policy in simulation. This initialization process completes within 20 minutes on a single NVIDIA RTX 4090 GPU. To enhance real-world training efficiency, we first performed hyperparameter tuning in a noisy simulation environment.

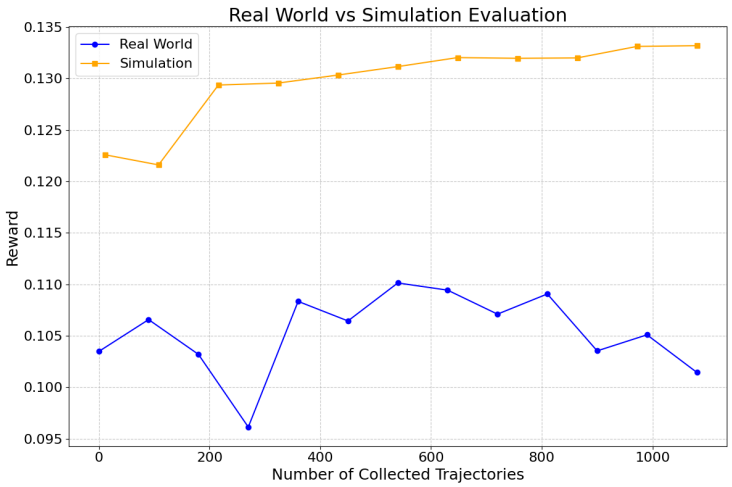


Figure 7: PPO training evaluation performance for simulation and real world.

We then conducted real-world PPO training. For real-world experiments, collecting a single trajectory takes approximately 10 seconds. We conducted two experimental trials. The first was paused after 180 trajectories because the policy output standard deviation was too high, leading to safety issues and inefficient exploration. In the second trial, after fixing this, we collected 1080 trajectories. Unfortunately, as shown in Figure 7, our results show no significant improvement in evaluation performance. We attribute this to two factors: 1) quantitative analysis of the collected data indicated that the noise variance in camera-based pose estimation, approximately 0.01, was comparable to the magnitude of potential reward gains estimated from simulation experiments, thereby obscuring the learning signal; and 2) the sample efficiency of training an open-loop network with PPO is inherently low. To make real-world RL effective, future work could focus on more precise pose estimation and more sample-efficient algorithms like model-based RL.

Currently, closed-loop control is not implemented due to significant latency and variance in joint state readings and base pose estimation. Developing a low-latency feedback control loop remains a critical direction for future performance enhancement.

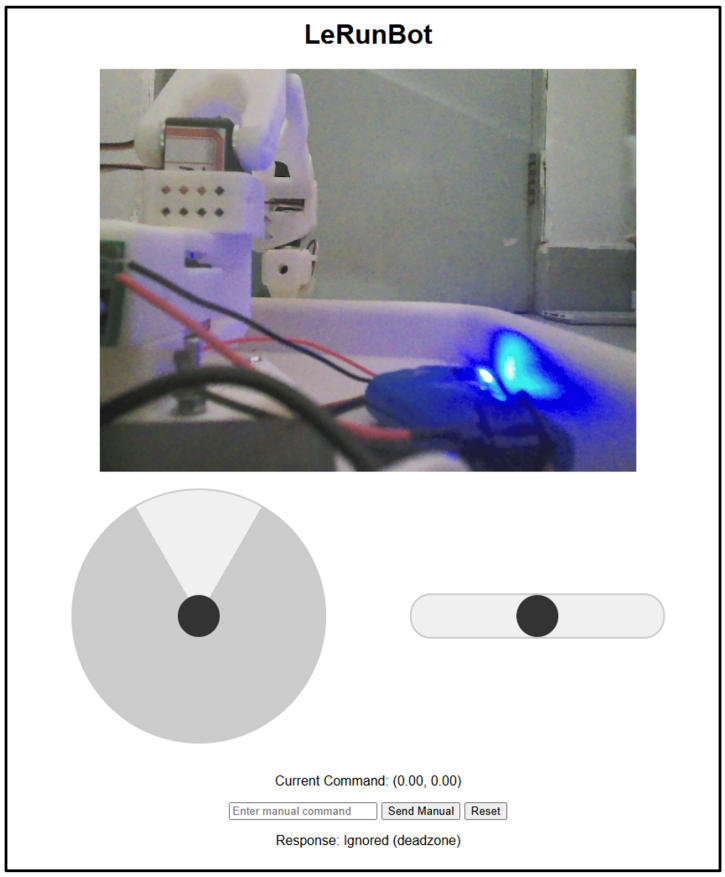


Figure 8: WebUI interface for Lerunbot.

5.3 High-level Control

The high-level controller serves as the user interface for issuing navigation commands, which are then translated into low-level execution directives. The system supports two high-level control strategies: *WebUI teleoperation* and *VLM-based instruction following*.

WebUI Teleoperation: We developed a WebUI that allows direct control. The interface streams real-time video from the robot and provides virtual joystick controls, mimicking a gamepad layout. Figure 8 shows the WebUI interface.

VLM-based Instruction Following: We integrated a Vision-Language Model (VLM) controller that processes natural language instructions. By using Visual Question Answering (VQA) capabilities, the controller reads the camera feed and user prompts to generate navigation commands via API calls to external services such as Qwen3-VL [1].

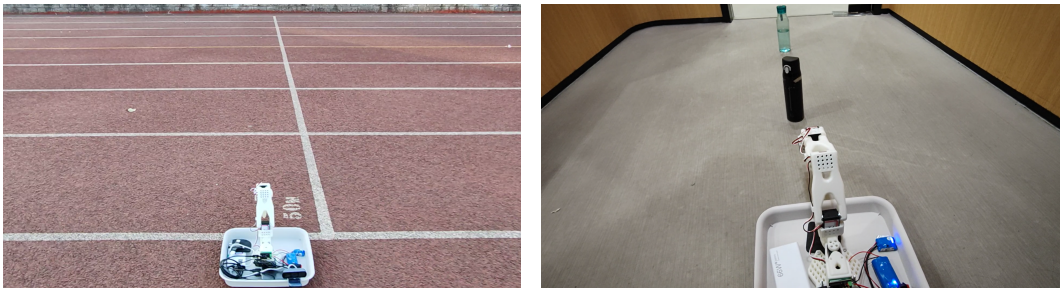
To evaluate this approach, we designed a visual navigation task where the robot is required to approach a target object placed in its field of view. The VLM is prompted to select one of three discrete actions: move forward, turn left, or turn right. We conducted experiments using Qwen3-VL-235B-A22B-Instruct [1] and GLM-4.5V [8]. Despite extensive prompt tuning, we found that standard non-reasoning VLMs lacked the necessary task understanding. For instance, the model would often correctly identify "the object is on the left", but paradoxically conclude "so I should turn right". While reasoning models could deduce the correct action, their inference latency was prohibitively high, often requiring a minute per decision. We therefore conclude that current open-access VLMs are not yet suitable for real-time high-level control in our system.

5.4 Real World Deployment

Ultimately, we adopted the planning-based strategy for low-level control and WebUI teleoperation for high-level control. To verify the reliability of this configuration and the robustness of the entire system, we conducted two designated tasks.

10-meter Shuttle Run: As shown in Figure 9a[1], in this task, Lerunbot is required to travel back and forth along a 10-meter straight line, which includes a U-turn. This experiment primarily tests the robot's maximum speed, battery endurance, and long-term heading stability against accumulated drift. We succeeded on the first attempt, completing the course in under 9 minutes.

Figure-8 Maneuver: As shown in Figure 9b[2], this task requires the robot to traverse a figure-eight path while avoiding two obstacles placed in the center. It evaluates the robot's steering capability, path tracking accuracy, and stability on complex trajectories. The robot successfully completed the task within two attempts.



(a) 10-meter Shuttle Run. (b) Figure-8 Maneuver.

Figure 9: Real-world deployment tasks.

6 Limitations and Future Work

While Lerunbot demonstrates a novel approach to robotic locomotion using a standard 6-DoF arm, several limitations remain that present opportunities for future research and development.

Closed-loop Control: Currently, the system lacks a robust closed-loop control mechanism due to latency and noise in state estimation. Future work could focus on developing low-latency feedback loops and improving sensor accuracy to enable real-time adjustments during locomotion.

Real-world RL Training: Our attempts at real-world reinforcement learning were hindered by noisy pose estimation and low sample efficiency. Future research could explore more precise localization techniques and sample-efficient algorithms, such as model-based RL, to enhance learning directly on physical hardware.

High-level Control with VLAs: We found that current Vision-Language Models (VLMs) are not yet suitable for real-time high-level control due to limitations in task understanding and inference latency. Future advancements in Vision-Language-Action models could enable more effective natural language instruction following.

7 Conclusion

In this project, we presented Lerunbot, an embodied AI system that endows a standard 6-DoF robotic arm with the ability to walk using its end-effector to propel itself. We detailed the hardware configuration, simulation environment, and reinforcement learning training process. We described the control architecture, including low-level and high-level controllers, and discussed various control strategies. Finally, we demonstrated Lerunbot's capabilities through real-world deployment tasks,

[1] Video recording: `https://cloud.tsinghua.edu.cn/f/e4ea4c74db844f45840f/`

[2] Video recording: `https://cloud.tsinghua.edu.cn/f/5da27139ddc449e2a915/`

highlighting its potential for future research in robotic locomotion and embodied AI. We also discussed the limitations of the current system and outlined directions for future work to enhance its performance and capabilities.

References

[1] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, Wenbin Ge, Zhifang Guo, Qidong Huang, Jie Huang, Fei Huang, Binyuan Hui, Shutong Jiang, Zhaohai Li, Mingsheng Li, Mei Li, Kaixin Li, Zicheng Lin, Junyang Lin, Xuejing Liu, Jiawei Liu, Chenglong Liu, Yang Liu, Dayiheng Liu, Shixuan Liu, Dunjie Lu, Ruilin Luo, Chenxu Lv, Rui Men, Lingchen Meng, Xuancheng Ren, Xingzhang Ren, Sibo Song, Yuchong Sun, Jun Tang, Jianhong Tu, Jianqiang Wan, Peng Wang, Pengfei Wang, Qiuyue Wang, Yuxuan Wang, Tianbao Xie, Yiheng Xu, Haiyang Xu, Jin Xu, Zhibo Yang, Mingkun Yang, Jianxin Yang, An Yang, Bowen Yu, Fei Zhang, Hang Zhang, Xi Zhang, Bo Zheng, Humen Zhong, Jingren Zhou, Fan Zhou, Jing Zhou, Yuanzhi Zhu, and Ke Zhu. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025.

[2] Albert Bou, Matteo Bettini, Sebastian Dittert, Vikash Kumar, Shagun Sodhani, Xiaomeng Yang, Gianni De Fabritiis, and Vincent Moens. Torchrl: A data-driven decision-making library for pytorch, 2023.

[3] Remi Cadene, Simon Alibert, Alexander Soare, Quentin Gallouedec, Adil Zouitine, Steven Palma, Pepijn Kooijmans, Michel Aractingi, Mustafa Shukor, Dana Aubakirova, Martino Russi, Francesco Capuano, Caroline Pascal, Jade Choghari, Jess Moss, and Thomas Wolf. Lerobot: State-of-the-art machine learning for real-world robotics in pytorch. `https://github.com/huggingface/lerobot`, 2024.

[4] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, 2018.

[5] Rob Knight, Pepijn Kooijmans, Remi Cadene, Simon Alibert, Michel Aractingi, Dana Aubakirova, Adil Zouitine, Russi Martino, Steven Palma, Caroline Pascal, and Thomas Wolf. Standard Open SO-100 & SO-101 Arms.

[6] Jianlan Luo, Zheyuan Hu, Charles Xu, You Liang Tan, Jacob Berg, Archit Sharma, Stefan Schaal, Chelsea Finn, Abhishek Gupta, and Sergey Levine. Serl: A software suite for sample-efficient robotic reinforcement learning, 2025.

[7] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017.

[8] V Team, Wenyi Hong, Wenmeng Yu, Xiaotao Gu, Guo Wang, Guobing Gan, Haomiao Tang, Jiale Cheng, Ji Qi, Junhui Ji, Lihang Pan, Shuaiqi Duan, Weihan Wang, Yan Wang, Yean Cheng, Zehai He, Zhe Su, Zhen Yang, Ziyang Pan, Aohan Zeng, Baoxu Wang, Bin Chen, Boyan Shi, Changyu Pang, Chenhui Zhang, Da Yin, Fan Yang, Guoqing Chen, Jiazheng Xu, Jiale Zhu, Jiali Chen, Jing Chen, Jinhao Chen, Jinghao Lin, Jinjiang Wang, Junjie Chen, Leqi Lei, Letian Gong, Leyi Pan, Mingdao Liu, Mingde Xu, Mingzhi Zhang, Qinkai Zheng, Sheng Yang, Shi Zhong, Shiyu Huang, Shuyuan Zhao, Siyan Xue, Shangqin Tu, Shengbiao Meng, Tianshu Zhang, Tianwei Luo, Tianxiang Hao, Tianyu Tong, Wenkai Li, Wei Jia, Xiao Liu, Xiaohan Zhang, Xin Lyu, Xinyue Fan, Xuancheng Huang, Yanling Wang, Yadong Xue, Yanfeng Wang, Yanzi Wang, Yifan An, Yifan Du, Yiming Shi, Yiheng Huang, Yilin Niu, Yuan Wang, Yuanchang Yue, Yuchen Li, Yutao Zhang, Yuting Wang, Yu Wang, Yuxuan Zhang, Zhao Xue, Zhenyu Hou, Zhengxiao Du, Zihan Wang, Peng Zhang, Debing Liu, Bin Xu, Juanzi Li, Minlie Huang, Yuxiao Dong, and Jie Tang. Glm-4.5v and glm-4.1v-thinking: Towards versatile multimodal reasoning with scalable reinforcement learning, 2025.

[9] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033. IEEE, 2012.